CPE 334 Introduction to Software Engineering in the Age of AI Agents

Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 5. Cloud Deployment and CI/CD Pipeline on GCP. **Score: ____/ 100**

Instructors: Assoc. Prof. Suthep Madarasmi, Ph.D. (Jogie) (suthep.mad@kmutt.ac.th)
Aj. Piyanit Ua-areemitr, Ph.D (Toey) (piyanit.wep@kmutt.ac.th)
Aj. Santawat Thanyadit Ph.D. (Job) (santawat.than@kmutt.ac.th)

TAs: Rachawipa Katippatee (Bom) (rachawipa.kati@gmail.com)
Kantapat Suwannahong (Bump) (kantapat.suwan@kmutt.ac.th)
Rattanachote Petpansri (Loogmoo) (rattanachote.petpa@kmutt.ac.th)
Prapatsorn Sangrod (Noon) (prapatsorn.sangr@kmutt.ac.th)
Supachok Deetaweesukh (Tik) (jedsadaporn.pann@mail.kmutt.ac.th)

Lab 5 briefly

Item	Details
Type	**Group lab.** Same team of two, same repository, and same application as your Group Project.
Starting point	Your accepted Sprint 1 release (Project Deliverable 1, due Sunday 8 November 2026).
Lab sessions	3 and 5 November 2026, with the TA walkthrough in Lecture 11 (Section 6).
Due	**Sunday 15 November 2026, 23:59**
Points	**100 lab points** (5% of the course grade).
Rubric	CI and quality gate 30 | Docker and reproducible configuration 20 | GCP deployment and persistence 35 | Repeatable release pipeline 15.
Required vs. recommended	**Required** items come from the CPE 334 Group Work Specification (Section 4) and are graded. **Recommended** items (Section 5) are what the Starter Kit and Student Guide already do; they are not scored separately.
Materials	**Student Guide: GCP Deployment and CI/CD (V1.1)** shows how to do each step, with videos on LEB2. **Lab 5 Starter Kit** has template files to adapt (Section 6).
GCP access	GCP Free Trial (USD 300 credit) on one member's personal Gmail account, as in Student Guide Episode 1 (Section 4.4).
Public repository	**Required (R-12).** Never commit .env, passwords, connection strings, or keys (Section 4.3).
Submit	One PDF per team on LEB2, named **lab5_sec<section>_<group name>.pdf** (Section 12).
Questions	Ask in the course Facebook group.

1. Software Delivery Increment in Lab 5

In Labs 1 to 4, TokTickIT ran only on your own computer. Lab 5 is a group lab: your team of two takes its accepted Group Project Sprint 1 release from a laptop to Google Cloud Platform (GCP) and adds a pipeline that checks every change and deploys the accepted ones. You keep the same application, GCP project, and pipeline for Lab 6, Lab 7, and the Final Project. By the end of Lab 5:

- the application is packaged with Docker and builds and runs from a clean clone;
- it runs on GCP at a public URL, with the UI, API, and database working together;
- its data survives an application restart and a redeploy;
- every relevant Pull Request runs the build and the automated tests, and a failed required check blocks the merge; and
- accepted releases deploy from main through a repeatable pipeline, and you can show which commit is running.

Both members take part in all of Lab 5: you may split tasks, but not into permanent roles, and either member must be able to explain any sampled release step (R-11). The default is a shared team score; suspected non-participation is referred to the instructor.

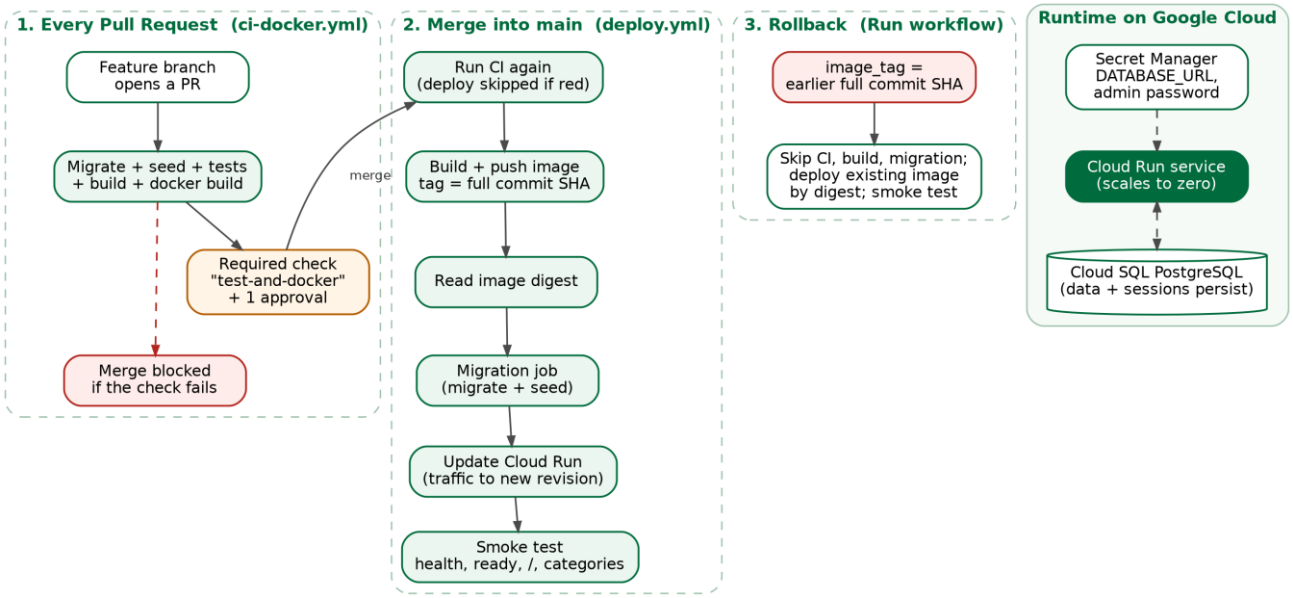


Figure 1. *Lab 5 delivery path using the reference stack. The stack and the rollback path (3) are recommended, not required (Section 5); the requirements are in Section 4.*

2. Lab 5 Learning Outcomes

- package a full-stack application into a container image that builds from a clean clone and reads its configuration from the environment;
- deploy the container to GCP with a persistent database, and show that data survives a restart and a redeploy;
- protect main so that only reviewed changes with passing build and test checks can be merged;
- release accepted changes from main through a repeatable pipeline and identify which commit is running; and
- use AI coding agents to build the pipeline while checking that every command, permission, and secret is safe.

3. Lab 5 Request from Stakeholder

> "Your Sprint 1 release works on your laptops, but nobody else can use it yet. Put it on Google Cloud so that we can open it in a browser at a real URL. Anything we enter must still be there after a restart and after your next release.
>
> From now on, a change may reach the live system only after it has been reviewed and has passed its tests, and every release must come from main through the same repeatable process. In Lab 6 a partner team will run user acceptance tests on your deployed system and report defects, and you will ship the fixes through this same pipeline.
>
> Keep costs low, and do not leave paid resources running when nobody needs them."

4. Required Work

Every requirement below comes from the CPE 334 Group Work Specification ("Spec"). The rubric in Section 12 grades these items only.

4.1. Requirements

ID	Requirement	Spec Section
R-01	Package the application with Docker. A fresh build from a clean clone produces a working container.	5.1, 5.3, 10
R-02	Read every setting from the environment. Document each environment variable in .env.example or deployment.md, and commit no real secret.	5.3, 10
R-03	Deploy on GCP at a public URL. The domain workflow works there through the UI, the API, and the database.	5.3, 10
R-04	Store data in a persistent database. Data survives an application restart and a redeploy, so nothing the app needs is kept only in the container.	5.1, 10
R-05	Run the build and the automated tests on every relevant Pull Request, with meaningful test results.	5.3, 10
R-06	Make the passing check required, so a failed check cannot enter the release path. Show it with a failing Pull Request from a temporary branch, and never break the released application.	5.3, 10
R-07	Deploy accepted releases from main through a repeatable pipeline: reviewed change, CI, merge, deployment. A documented manual approval inside the pipeline is allowed. A separate cloud staging environment is not required.	5.3, 8.3, 10
R-08	Identify the commit of each release (tag or commit SHA), and document the release procedure so it can be repeated.	8.3, 10

ID	Requirement	Spec Section
R-09	Use Issues and a project board, feature/fix branches, Pull Requests into staging reviewed by your teammate, and a release Pull Request from staging into main. Authors do not approve their own Pull Requests, and every review states a specific check, question, finding, or reason for approval.	5.3, 8.3
R-10	Label Lab 5 Issues and Pull Requests (for example lab5) so the evidence is clearly separate from unrelated feature work. Keep the history: do not delete or rewrite commits, contract revisions, or CI runs. CI stays mandatory after Lab 5, and Lab 7 uses its run history.	8.3
R-11	Both members take part in requirements, testing, AI-agent supervision, implementation, review, and deployment. Record contributions through Issues, commits, and Pull Request authorship and review; for pair work, name both members in the Issue or Pull Request. Equal commit counts are not required.	2
R-12	Make the repository public and keep it public until grading ends, so the TAs and your Lab 6 partner team can open the code, Issues, Pull Requests, and workflow runs. TAs are not added as collaborators to private repositories. A public repository also lets the R-06 ruleset work on GitHub Free.	11

4.2. Not Required in Lab 5

A separate cloud staging environment, Kubernetes or infrastructure-as-code tools, custom domains, load balancers, multi-region deployment, canary or blue/green releases, monitoring dashboards, load testing, real payment or messaging services, and features outside your approved Sprint 1 scope.

4.3. Required Files

File	Content
Dockerfile, .dockerignore	Builds the application image. Add docker-compose.yml if you run the app locally with a database.
.env.example	Every environment variable the app reads, with placeholder values only.
CI and deploy workflows	For example .github/workflows/ci-docker.yml and deploy.yml, or your equivalent.
docs/labs/lab-05/deployment.md	Your stack and any change from the reference; each environment variable (purpose, secret or not, where its value comes from); the commands to build, test, run, migrate, and seed; GCP project, region, and resource names; and the release procedure, including how to find the commit that is running.

Your repository is public. Never commit a secret.

Anyone can read and copy what you push, and bots scan public GitHub repositories for keys within minutes. Never commit or push:

- a .env file with real values (keep .env in .gitignore and commit only .env.example);
- passwords, connection strings such as DATABASE_URL, API keys, or service account key files; or
- screenshots or logs that show any of them.

Check git status and the diff before every commit. If a secret is pushed, deleting the file is not enough, because it stays in the Git history. Change the secret at once (for example the database password and its Secret Manager version), redeploy, and tell a TA.

4.4. GCP Access

The Spec leaves the GCP access method to this handout (Spec 5.3). Follow Student Guide Episode 1:

- one member signs up for the GCP Free Trial (USD 300 credit) with a personal Gmail account that has never used it, not a @mail.kmutt.ac.th account (1.1);
- that member creates one project for Lab 5, Lab 6, Lab 7, and the Final Project, for example cpe334-sec1-team07 (1.2), and adds the teammate in IAM with the Editor role (1.3); and
- nobody pays a prepayment or clicks Upgrade. If GCP asks for payment or says the billing account is closed, take a screenshot and tell a TA.

5. Recommended Practices

The items below are not in the Spec, so they are not scored separately. The Starter Kit and the Student Guide already follow them, and they are the shortest safe path to R-01 to R-12. If your team chooses differently, write the choice and the reason in deployment.md.

Recommended practice	Student Guide / kit
Cost control (strongly recommended): Cloud SQL is billed for every hour it exists. Create an alerts-only budget before any resource, set Cloud Run minimum instances to 0 and maximum to 3 or fewer, use the smallest Cloud SQL tier with storage auto-increase off, pause when idle, and tear down after the course.	Episodes 1, 5.4, 9, 10
Secret scanning push protection: turn it on in the public repository so GitHub blocks a push that contains a key or password. If a secret is ever pushed, change it at once.	R-02, R-12
Reference stack: Artifact Registry, Cloud Run, Cloud SQL for PostgreSQL, Secret Manager, and GitHub Actions.	Episode 0.2–0.4
Keep sessions and uploaded files in the database, not in the container (this helps R-04).	Episode 2
Seed demo accounts for each role so the Lab 6 partner team can test. Keep their passwords out of the code and never show them on screen; share them with the partner team privately.	Episode 2
Keep runtime secrets in Secret Manager and pipeline secrets in GitHub Actions secrets.	Episodes 5, 6

Recommended practice	Student Guide / kit
Sign in from GitHub with Workload Identity Federation instead of a JSON key; give service accounts and workflows only the permissions they need.	Episode 6
Tag each image with the full commit SHA and deploy it by digest (the simplest way to meet R-08).	Episode 7
Run migrations in a separate job before the service is updated, and keep them additive.	Episodes 5, 7
Run an automated, read-only smoke test against the live URL after every deploy.	Episode 7
Roll back by redeploying an earlier image tag, then roll forward.	Episode 8
Pin GitHub Actions to a full commit SHA and let Dependabot propose updates.	Kit README

6. Supporting Materials and Lecture Walkthrough

Material	Use it for
Student Guide: GCP Deployment and CI/CD (V1.1)	Step-by-step Episodes 0 to 10. Each lists its commands, a Done check, and an "If something fails" table. Videos: LEB2 > Learn It > Student Guide: GCP Deployment and CI/CD (Video).
Lab 5 Starter Kit (toktickit-lab5-starter)	The TokTickIT deployment as a worked example, without the application code. Its README lists each file and what to change for another stack. Treat every file as a template to adapt.

In Lecture 11 the TAs demonstrate the full path on TokTickIT in both the Thai and the English sections: signing in to GCP, deploying from a local machine, connecting a persistent database, deploying an approved change through CI/CD, and showing a failing check that blocks a merge. If the guide or the kit disagrees with this handout, follow this handout and tell a TA.

7. Lab 5 Execution Steps

Work through the Student Guide in order. Each step has its own GitHub Issue (the guide numbers them #1 to #8).

#	What to do	Guide	Done when
1	Start the GCP Free Trial, create the project, and add your teammate (Section 4.4). Recommended: create the budget alert first.	Ep. 1	Billing → Credits shows the credit and both members can open the project.
2	Make the app cloud-ready and write the first deployment.md.	Ep. 2	Settings come from the environment and nothing the app needs is kept only in the container (R-02, R-04).
3	Containerize the app.	Ep. 3	A fresh build runs with its database (R-01).

#	What to do	Guide	Done when
4	Add CI on Pull Requests.	Ep. 4	A Pull Request shows the build and test checks (R-05).
5	Deploy to GCP for the first time.	Ep. 5	The public URL opens and you can sign in (R-03).
6	Connect GitHub to GCP and set the merge gate.	Ep. 6	A failing Pull Request from a temporary branch cannot be merged (R-06).
7	Add the deploy workflow; release a visible change through staging into main.	Ep. 7	The URL shows the change and you can show which commit is running (R-07, R-08).
8	Check persistence. Recommended: roll back and roll forward.	Ep. 8	Data created earlier survives a restart and a redeploy (R-04).
9	Finish deployment.md and the PDF.	Sec. 4.3, 12	Every rubric part has a row in the evidence index.

8. Required Checks

Record each check with the file or test name, the command or CI run, the result, and the tested commit (Spec 11).

Check	What it shows	Covers	Evidence
C-01	A normal Pull Request runs the build and all automated tests, and they pass.	R-05	CI run link and test summary
C-02	A Pull Request from a temporary branch with one failing test cannot be merged.	R-06	PR link; failed check and blocked merge
C-03	A fresh clone builds and runs with only .env.example values filled in.	R-01, R-02	Commands and output
C-04	The domain workflow works on the public URL: sign in, the main business action, and one rule rejection or access restriction.	R-03	Screenshots with the URL visible
C-05	A record created before a restart and a redeploy is still there afterwards.	R-04	Before and after screenshots
C-06	A reviewed change goes from staging into main and is deployed, and the running revision matches its commit.	R-07 to R-10	PR links, pipeline run, GCP revision

9. Team Workflow and AI Agents

Similar to Labs 2 to 4. Each Issue has its own feature branch and enters staging through a Pull Request reviewed by your teammate. staging enters main through a release Pull Request. Only main deploys, so every merge into main is a release. Cloud setup done in the Console still gets an Issue; close it with a screenshot. When you use AI agents:

- read every gcloud command or console change an agent proposes before it runs, and never let an agent change billing, IAM, or databases on its own;
- never paste a secret, password, connection string, or key file into an agent chat; and
- when a deploy fails, give the agent the log excerpt and ask for the cause before you ask for a fix.

10. Common Problems

Each Student Guide episode ends with an "If something fails" table, and kit docs/gcp-cicd-guide.md Section 8 adds more. Read the log of the failed step before changing anything. Not covered there:

Symptom	Likely cause and fix
Enabling an API fails with FAILED_PRECONDITION or UREQ_PROJECT_BILLING_NOT_OPEN.	Billing is not open. Check billingEnabled=true and the billing account, then tell a TA.
gcloud sql instances create rejects db-f1-micro for PostgreSQL 16.	Add --edition=enterprise.
A step fails right after an account or role was created.	IAM changes take a short time to spread. Wait a minute and retry.
A secret leaked, and a new Secret Manager version changed nothing.	Change the password where it is used (for example the database user) as well, then redeploy.

11. Definition of Done for Lab 5

Tick every box before you submit.

☐ a fresh clone builds and runs; every setting comes from the environment, and no secret is in the repository, the image, the logs, or the PDF;

☐ the public URL serves the latest release from main, the domain workflow works there, and you can show which commit is running;

☐ data survived a restart and a redeploy;

☐ CI runs on every Pull Request into staging and main, and a failed required check blocks the merge;

☐ the latest release reached main through reviewed Pull Requests and was deployed by the pipeline;

☐ the repository is public (R-12);

☐ deployment.md and the evidence index are complete, and either member can explain any sampled step; and

☐ **the PDF is named lab5_sec<section>_<group name>.pdf.**

12. Submit One PDF File

Each team submits one concise PDF on LEB2, named **lab5_sec<section>_<group name>.pdf** (for example lab5_sec1_bytebusters.pdf), by **Sunday 15 November 2026, 23:59**. The PDF is an evidence index: link to the repository, Pull Requests, workflow runs, and the live service instead of repeating them. Use the headings **"Answer Part 0" to "Answer Part 4"** in order. Keep the service running until the instructor announces the end of grading.

Links, explanations, and screenshots. For each item, give a direct link and one or two sentences that explain what it shows. Add a screenshot only when it is needed:

- use a screenshot only for what a link cannot show on its own, such as a blocked merge button, a GCP Console setting, or the app before and after a redeploy; leave out screenshots that repeat a link;
- give every screenshot a numbered caption that says what it shows, which check or requirement it proves, and the commit, run, or URL it comes from, for example: **"Figure 3. C-02 (R-06): PR #14 from a temporary branch; the required check test-and-docker failed and merging is blocked. Commit a1b2c3d."**
- crop to the part that matters and keep the text readable at 100% zoom; do not shrink a full-screen capture to fit, and use the full page width for screenshots with small text. A box or arrow on the key part helps; and
- place each screenshot under its Answer Part next to its link, and hide or blur any secret value before you insert it.

How marks are given (Spec 11): TAs sample requirements, tests, Issues, Pull Requests, and deployed journeys. Marks reward observable behavior, traceability, meaningful tests, and credible execution evidence. Document length, prompt count, commit count, and visual polish alone do not earn marks, so keep the PDF short.

Part	Points	Required Submission Evidence
0. Team and Evidence Index (required, not scored separately)	–	Team number and topic; both students' names, student IDs, and GitHub usernames; public repository link, submitted tag or commit, public URL, and the commit it is serving; a short contribution table with Issue and PR links; the evidence index (example below).
1. CI and Quality Gate	30	The CI workflow file; a passing Pull Request with meaningful build and test results (C-01); the required-check setting for main; the failing Pull Request from a temporary branch with the blocked merge (C-02).
2. Docker and Reproducible Configuration	20	Dockerfile and .dockerignore; a fresh build and the running container with its database (C-03); .env.example and the environment variable table in deployment.md showing that no secret is committed.
3. GCP Deployment and Persistence	35	Public URL; the domain workflow on the live URL (C-04); UI, API, and database connectivity; the GCP service and database; data before and after a restart and a redeploy (C-05).

Part	Points	Required Submission Evidence
4. Repeatable Release Pipeline	15	One labelled change from Issue to Pull Request into staging with a specific teammate review, release Pull Request into main, CI, and deployment; the pipeline run and the running revision showing the release commit (C-06); the release procedure in deployment.md.
Total	**100**	

Evidence index. At least one row per rubric part (Spec 11):

Rubric part	Artifact / direct link	IDs / version	Verification result
1. CI and quality gate	PR #14 and its Actions run link	C-02, commit a1b2c3d	Check failed; merge blocked
3. GCP deployment	Public URL; persistence screenshots	C-05, release e4f5a6b	Record present after restart and redeploy

Format the PDF like this:

Answer Part 0: Team and Evidence Index

[Links, short explanations, and captioned screenshots]

Answer Part 1: CI and Quality Gate

[Links, short explanations, and captioned screenshots]

Answer Part 2: Docker and Reproducible Configuration

[Links, short explanations, and captioned screenshots]

Answer Part 3: GCP Deployment and Persistence

[Links, short explanations, and captioned screenshots]

Answer Part 4: Repeatable Release Pipeline

[Links, short explanations, and captioned screenshots]

สรุปภาษาไทย (Thai Summary)

ส่วนนี้สรุปสาระสำคัญของ handout เป็นภาษาไทย หากข้อความไม่ตรงกัน ให้ยึดฉบับภาษาอังกฤษด้านบน

หัวข้อ	รายละเอียด
ประเภทงาน	งานกลุ่ม 2 คน ใช้ทีม, repo และแอปเดียวกับ Group Project โดยเริ่มจาก Sprint 1 release ที่ผ่านการยอมรับแล้ว
กำหนดส่ง	**วันอาทิตย์ที่ 15 พฤศจิกายน 2569 เวลา 23:59 น.**
คะแนน	100 คะแนน (5% ของเกรด)
วิธีส่ง	ส่ง PDF 1 ไฟล์ต่อกลุ่มทาง LEB2 ชื่อไฟล์ **lab5_sec<1/31>_<ชื่อกลุ่ม>.pdf**
การเข้าใช้ GCP	สมาชิก 1 คนสมัคร GCP Free Trial (เครดิต USD 300) ด้วย Gmail ส่วนตัว ห้ามใช้บัญชี @mail.kmutt.ac.th ห้ามจ่าย prepayment และห้ามกด Upgrade (Student Guide Episode 1)
เอกสารประกอบ	- Student Guide: GCP Deployment and CI/CD (V1.1) - Lab 5 Starter Kit เป็นไฟล์ตัวอย่างให้นำไปปรับใช้
ติดต่อ / ถามปัญหา	กลุ่ม Facebook ของรายวิชา

สิ่งที่ต้องทำ (Required)

ID	สิ่งที่ต้องทำ
R-01	แพ็กแอปด้วย Docker เมื่อ build ใหม่จาก clean clone แล้วต้องรันได้
R-02	ค่าตั้งค่าทุกตัวอ่านจาก environment มี .env.example และไม่ commit secret จริง
R-03	deploy บน GCP มี URL สาธารณะ และ domain workflow ทำงานครบทั้ง UI, API และ database
R-04	ใช้ฐานข้อมูลถาวร ข้อมูลต้องยังอยู่หลัง restart และหลัง redeploy
R-05	ทุก PR ที่เกี่ยวข้องต้องรัน build และ automated tests ที่มีความหมาย
R-06	ตั้ง required check ให้ PR ที่ check ไม่ผ่าน merge ไม่ได้ สาธิตด้วย temporary branch และห้ามทำให้แอปที่ release แล้วพัง
R-07	deploy จาก main ผ่าน pipeline ที่ทำซ้ำได้ (review ⟶ CI ⟶ merge ⟶ deploy) มี manual approval ใน pipeline ได้
R-08	ระบุได้ว่าแต่ละ release มาจาก commit ไหน และเขียนขั้นตอน release ที่ทำซ้ำได้ไว้ใน deployment.md
R-09	ใช้ Issue, project board และ feature/fix branch เปิด PR เข้า staging ให้เพื่อนร่วมทีม review แบบเจาะจง แล้วเปิด release PR จาก staging เข้า main ผู้เขียน PR ห้าม approve PR ของตัวเอง
R-10	ติด label lab5 ที่ Issue และ PR ห้ามลบหรือเขียนทับ commit, contract revision และ CI run
R-11	สมาชิกทั้งสองคนมีส่วนร่วมทุกด้าน และบันทึกการมีส่วนร่วมผ่าน Issue, commit และ PR
R-12	repo ต้องเป็น public จนกว่าจะตรวจเสร็จ ห้าม commit ไฟล์ .env รหัสผ่าน connection string หรือ key หากเผลอ push secret ขึ้นไป ต้องเปลี่ยน secret ทันทีและแจ้ง TA

ข้อแนะนำ (Recommended): ควรตั้ง budget alert ก่อนสร้าง resource ทุกครั้ง เพราะ Cloud SQL คิดเงินทุกชั่วโมง

รูปแบบ PDF: ใช้หัวข้อ Answer Part 0 ถึง 4 ตามลำดับ ใส่ลิงก์พร้อมคำอธิบายสั้น ๆ เป็นหลัก ใส่ภาพเฉพาะที่จำเป็น ทุกภาพต้องมี caption ที่มีเลขกำกับ และบอกว่าพิสูจน์ check ข้อไหน ข้อความในภาพต้องอ่านได้ที่ซูม 100% และต้องไม่เห็น secret